

GPU

Outline

- GPU: Graphics Processing Unit
- GPGPU: General Purpose GPU

Graphics Processing Unit

GPU: Graphics Processing Unit

- GPU: designed to accelerate 3D rendering
 - Input: A description of a scene
 - 3D surface geometry (e.g., triangle mesh)
 - Surface materials, lights, etc.
 - Output: An image of the scene
 - Simple definition of rendering
 - Computing how each triangle in 3D mesh contributes to appearance of each pixel in the image

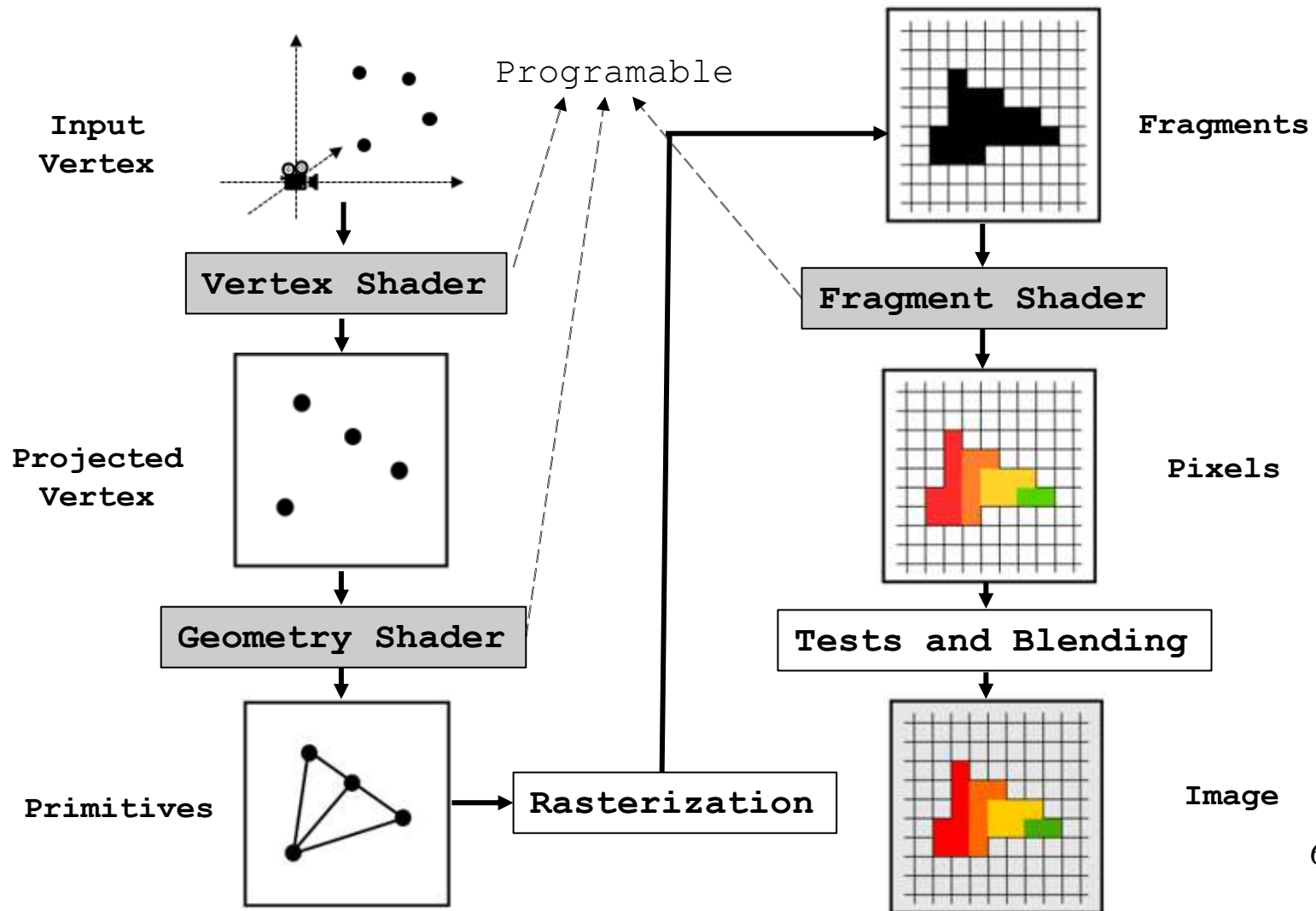


Rendering Complex 3D Scenes in Real-time



Unreal Engine 5

Steps of Rendering an Image

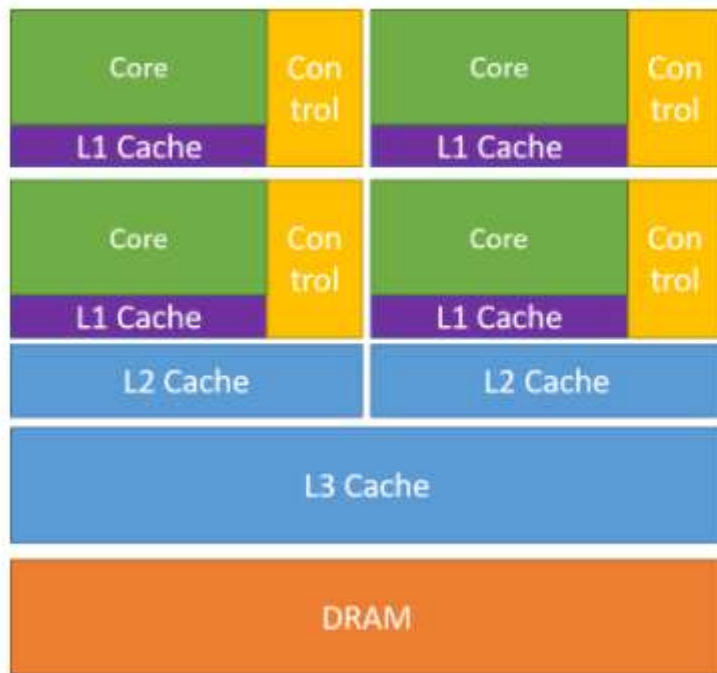


Each Step Iterates over every “Element”

- Shaders are mini-programs that operates on every element
 - e.g., Vertex shader operates on each vertex, and geometry shader operates on each triangle
- Modern 3D-games need to render **millions of triangles** per frame
 - 120 fps = billions of triangles / s
 - CPU cannot handle such workload in real-time.

GPU for 3D Rendering

- GPUs provide many small cores for efficient parallel execution of shader programs



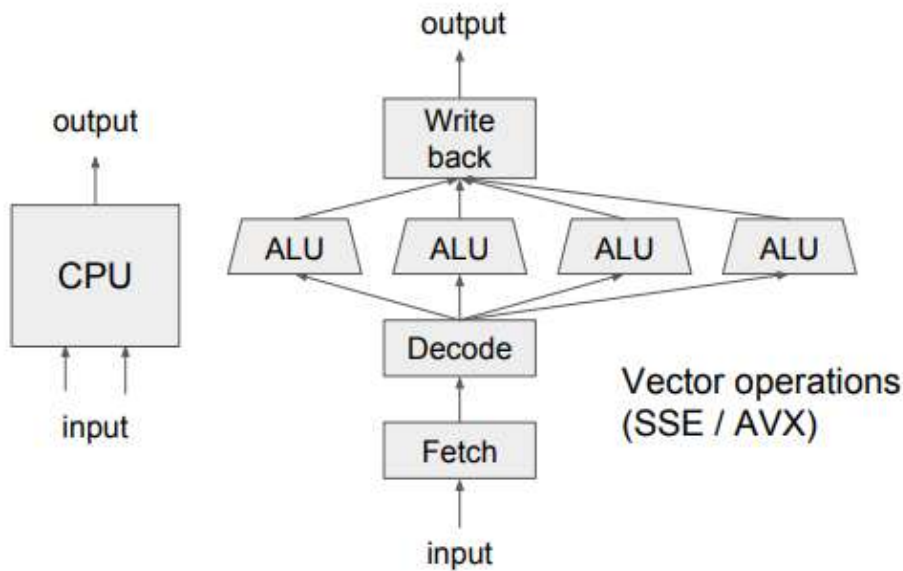
CPU



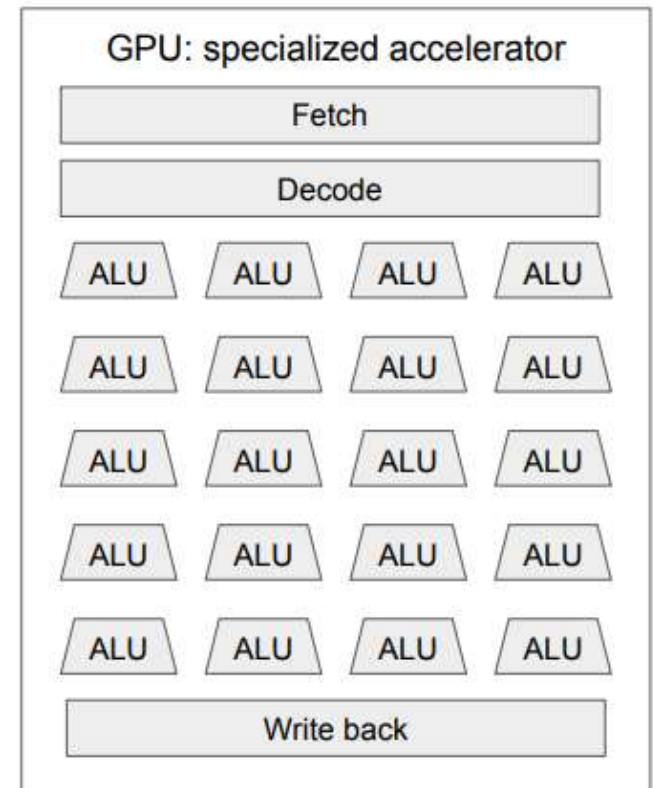
GPU

CPU vs. GPU

CPU vs GPU

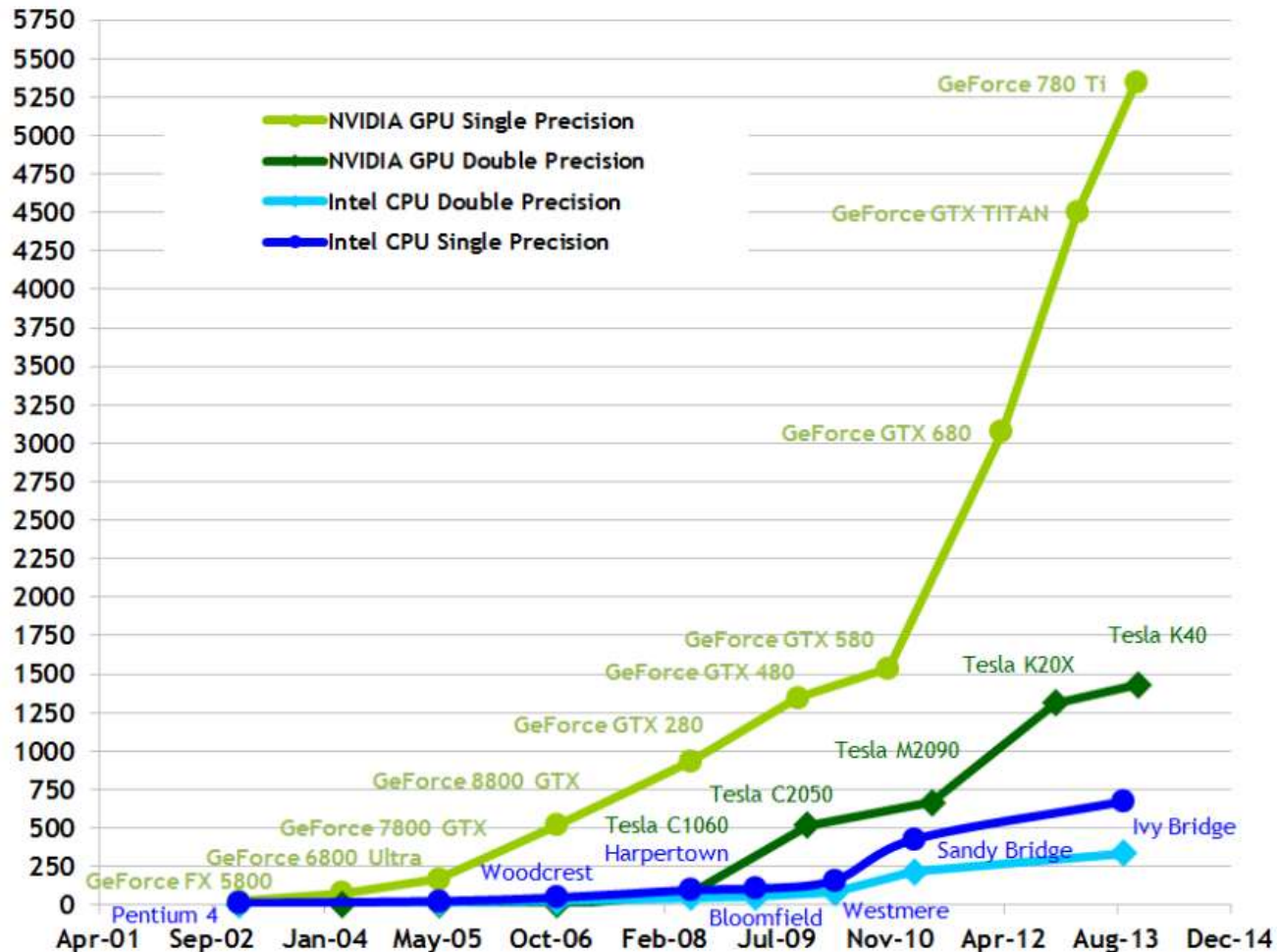


Vector operations
(SSE / AVX)



GPU Floating Point Performance (v.s. CPU)

Theoretical GFLOP/s



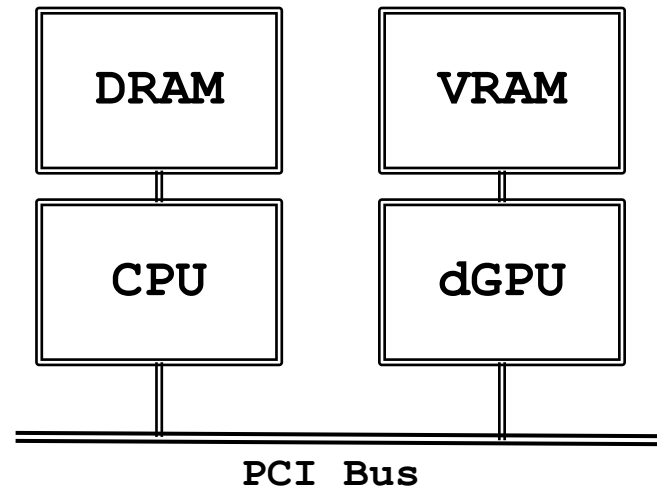
General Purpose GPU

Modern GPUs are also used for ...

- Machine learning / AI
- Scientific computing
- Data analytics

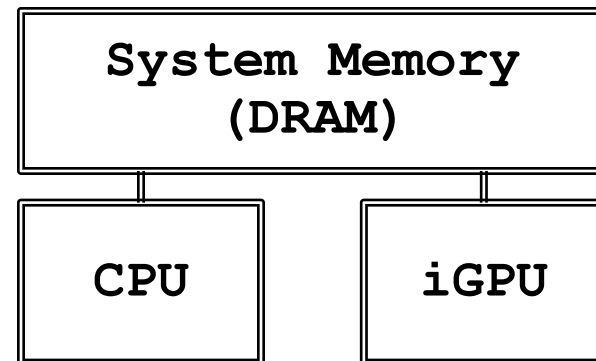
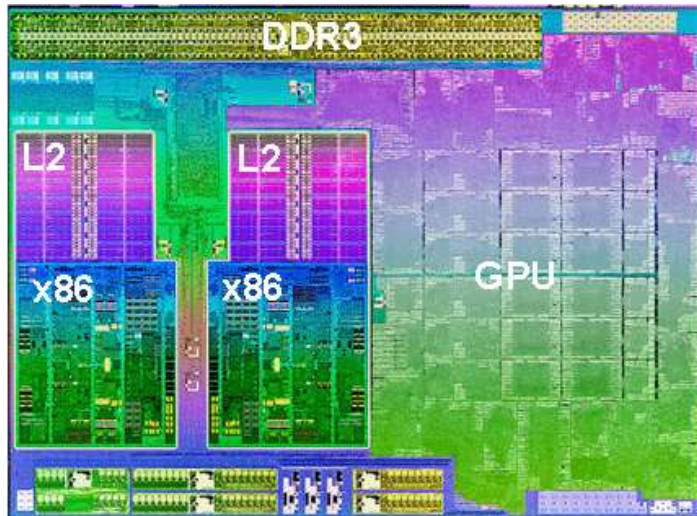
Discrete vs. Integrated GPUs

- Discrete GPUs (dGPUs)
 - Used in desktops and servers
 - Separate chip with dedicated VRAM
 - Connect to CPU via PCIe
 - More computational power & memory bandwidth



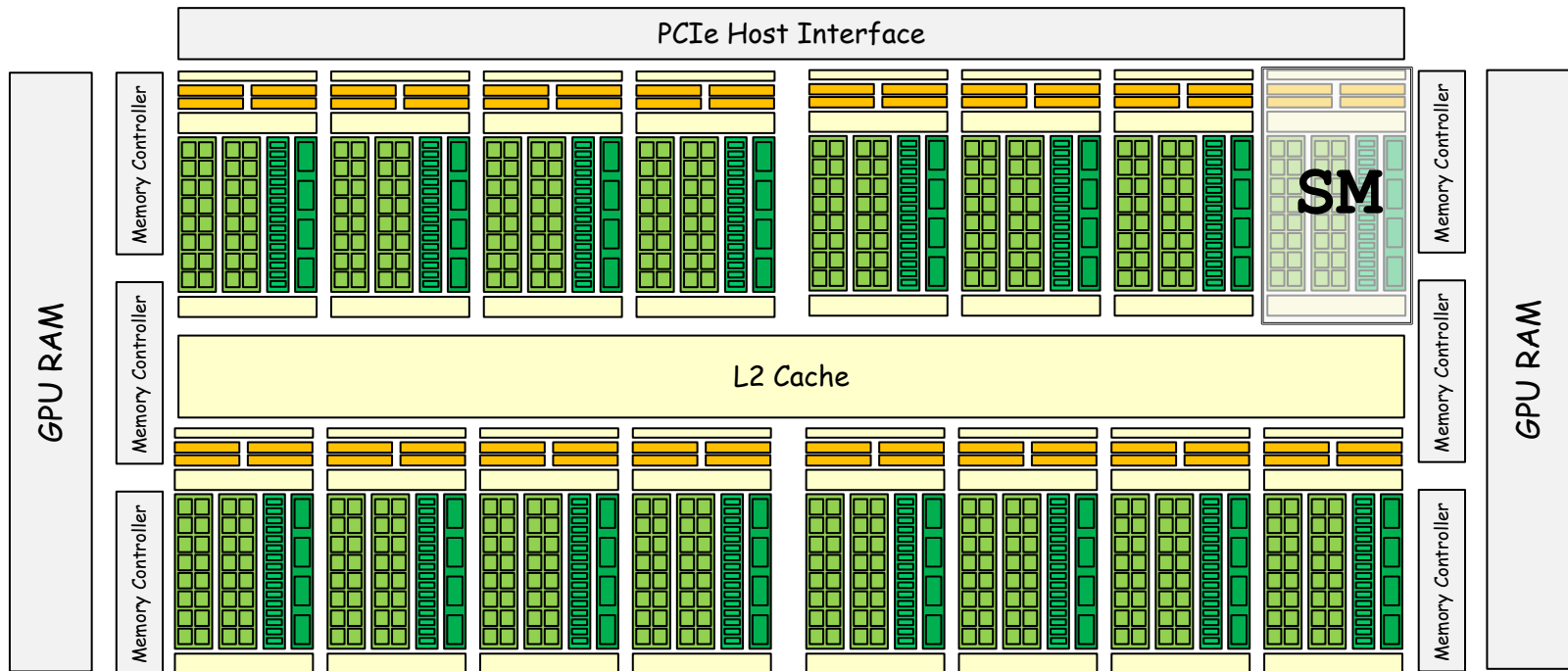
Discrete vs. Integrated GPUs

- Integrated GPUs (iGPUs)
 - Used in mobile devices (e.g., phones, laptops)
 - Share system memory with CPU
 - Less power (energy)



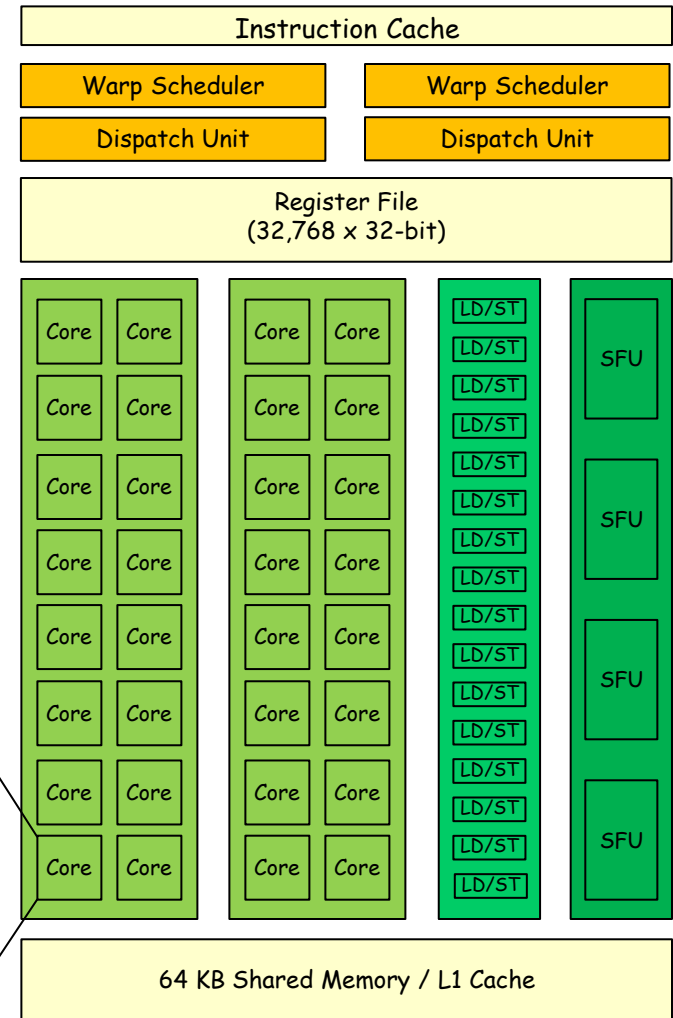
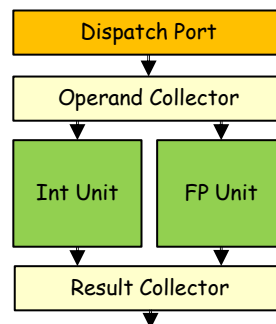
GPU Architecture Basis - NVIDIA GPU

- NVIDIA GPU consists of many streaming multiprocessors (SM)
 - SMs share a global L2 Cache



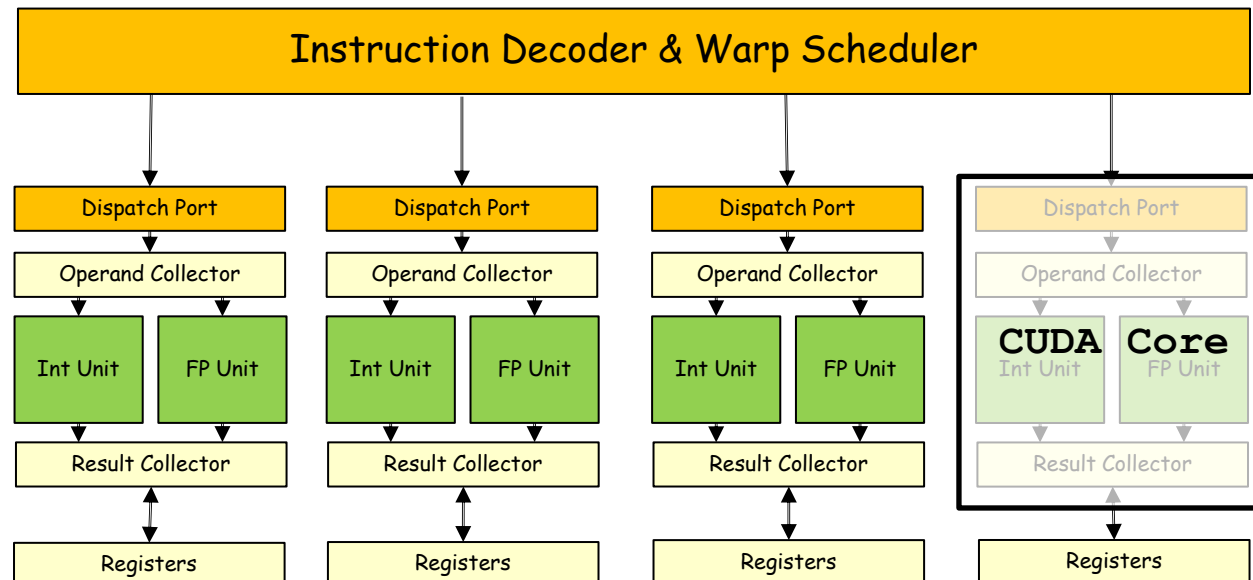
Streaming Multiprocessor

- SMs are analogous to CPU cores
 - SM is the basic computational unit
 - Consider an SM as a single vector processor
 - Composed of multiple
 - CUDA "Cores"
 - Load/Store (LD/ST) units
 - Special function units (SFUs)
- CUDA Core contains ALUs
 - Integer ALU
 - Floating-point ALU



SIMT Execution Model

- Single Instruction Multiple Threads
 - The instruction only decodes once
 - All "cores" execute the same instructions
 - But operate on their own private registers
 - A SIMT group names a "warp" (32 threads)

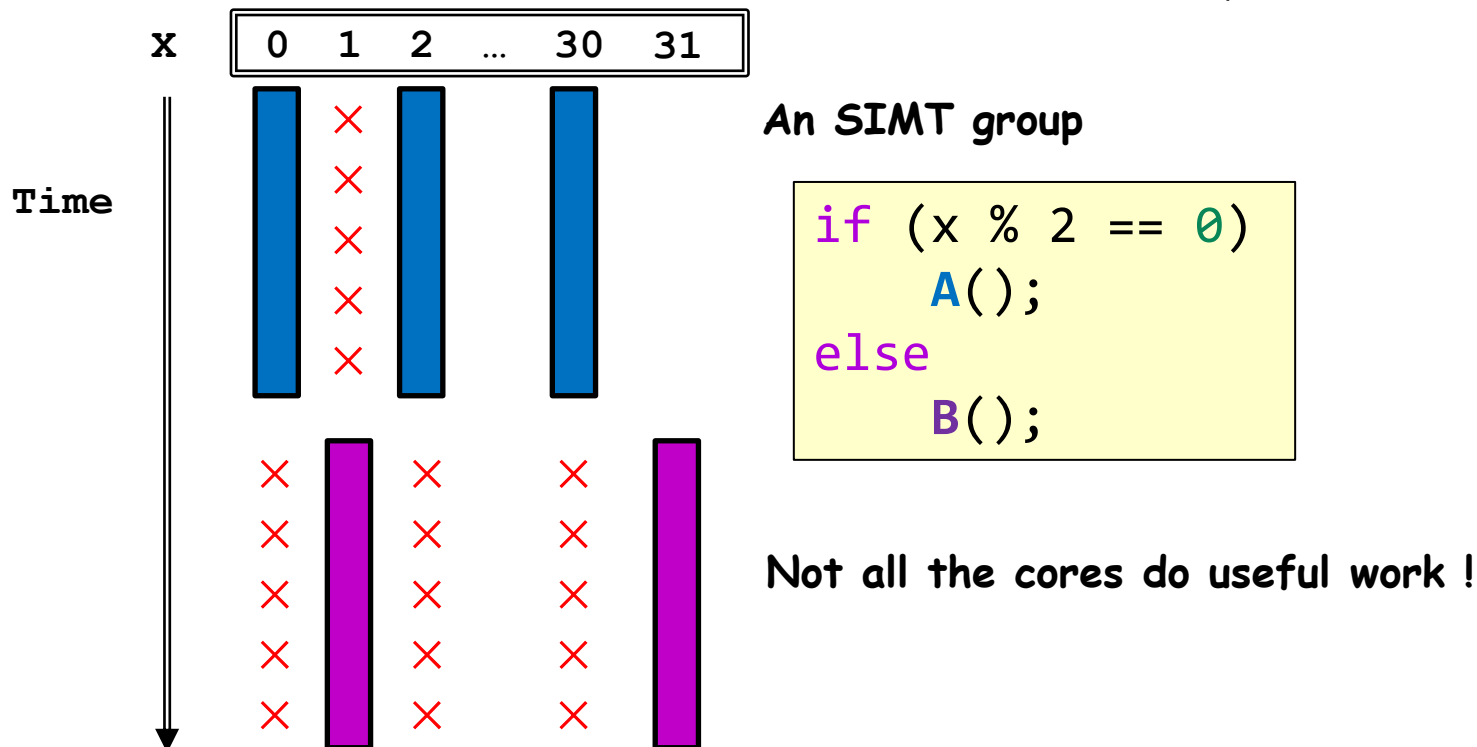


SIMT Execution Model

- An SM can execute many warps simultaneously
 - E.g., Fermi GPU has 2 warp schedulers per SM
- An SM can execute more warps concurrently
 - SM can switch the current executing warps
 - Switch out the inactive warps (e.g., blocked on memory I/O)
 - Switch in the warps that are ready for next instruction
 - Each thread stores its own state in its registers persistently
 - So the warp switching has zero cost

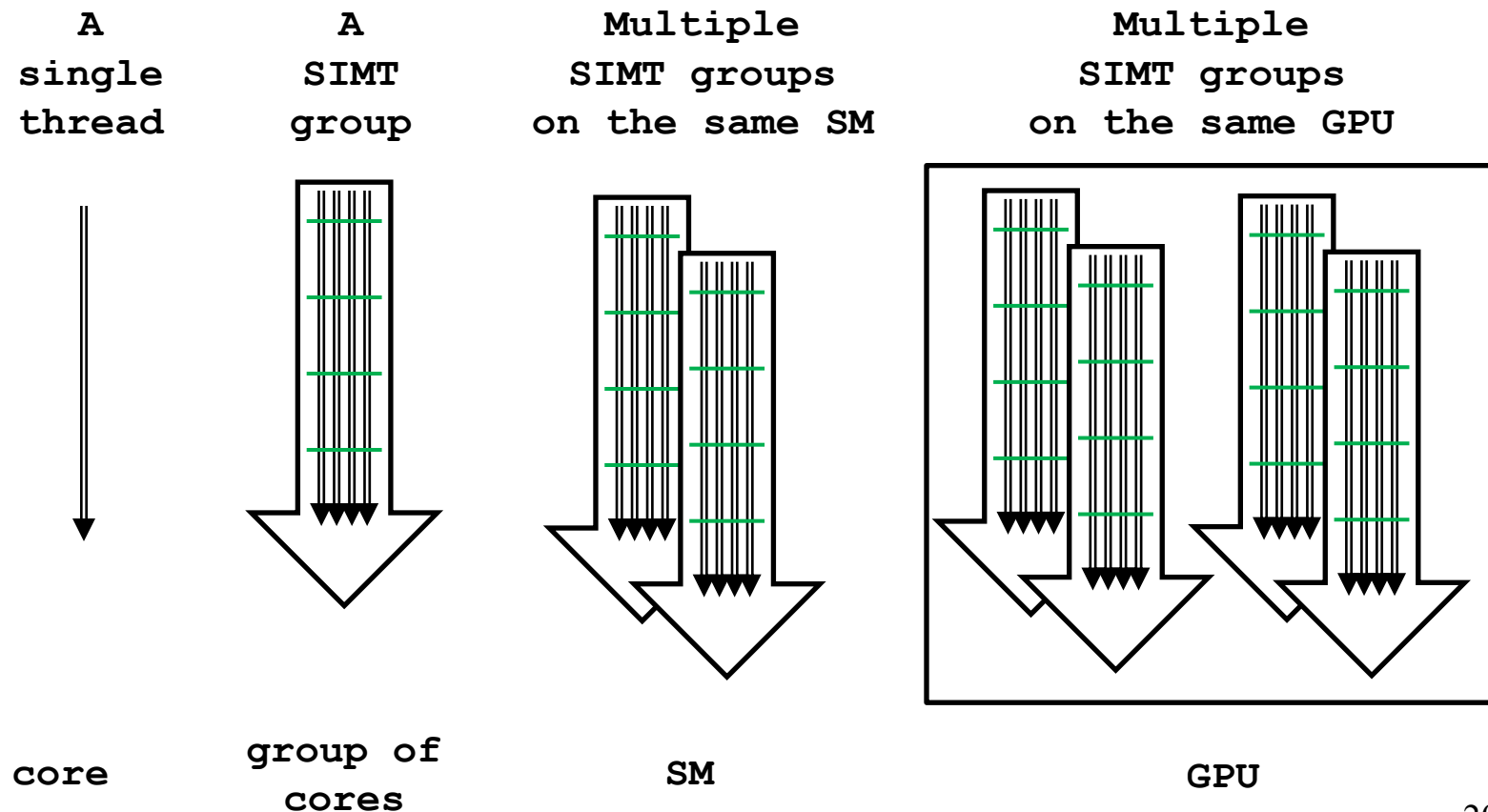
SIMT Execution Model - Thread Divergence

- Threads in an SIMT group may diverge after branch
 - Some cores are **disabled** for some instructions if they need to execute the other branch
 - The disabled cores are wasted for these cycles



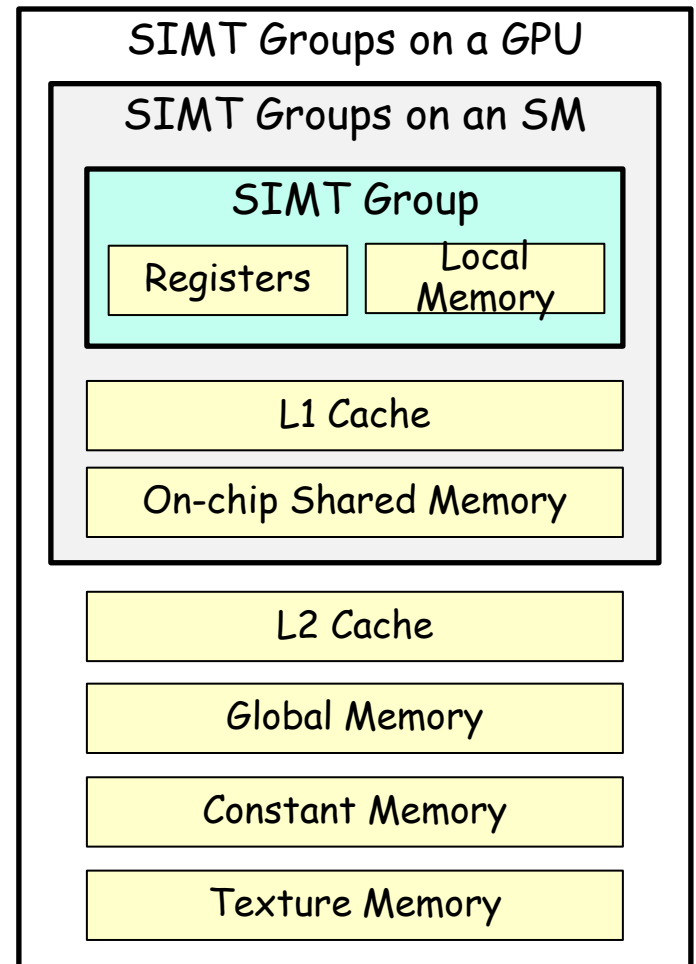
SIMT Execution Model

- Together, GPU has a nested thread hierarchy



GPU Memory Model

- There are many types of GPU memory, each of which has special characteristics that makes it useful
 - Size
 - Latency
 - Bandwidth
 - Readable/Writable
 - Optimal Access Patterns
 - Accessibility (share scope)



GPU Memory Model

Type	Read Latency (Cycles)	Size	R/W	Share Scope
Register	1	128 KB per SM	R/W	Thread
Local Memory	400 ~ 600	-	R/W	Thread
L1 Cache	1 ~ 32	192 KB per SM	Transparent	SM
Shared Memory	1 ~ 32	192 KB per SM	R/W	SIMT groups
L2 Cache	32 ~ 64	6 MB	Transparent	GPU
Global Memory	400 ~ 600	GBs	R/W	GPU
Constant Memory	100 ~ 200	KBs	Read-only	GPU
Texture Memory	400 ~ 600	-	Read-only	GPU

- Local memory is logical, similar to the stack of CPU, while it is physically global memory
- Constant memory is also global memory, while it supports special read/write instructions and caching mechanism
- L1/L2 is not programmable, similar to that of CPU

GPU Programming Languages

- For Graphics
 - Languages: GLSL, HLSL
 - Frameworks: OpenGL, Vulkan, DirectX
- For General Purpose Computing
 - OpenCL: an open standard
 - CUDA: for NVIDIA GPUs only
 - HIP: for NVIDIA and AMD GPUs
 - SYCL: for Intel GPUs

CUDA Programming Basis

- **Compute Unified Device Architecture**
 - A "C-like" language to express programs that run on GPUs
 - Introduced in 2007 with NVIDIA Tesla architecture
 - Only runs on NVIDIA GPUs
- **Driver:** low-level software that controls the device
- **CUDA** provides a lot of toolkits for GPU developments
 - NVCC: compiler for CUDA programs
 - CUDA Driver/Runtime: execution runtime
 - CUBLAS/CUDNN: C++/C libraries for computing

Terminology

- Host: the CPU
 - Host memory: CPU memory
 - Host code: code runs on CPU
- Device: the GPU
 - Device memory: (discrete) GPU memory
 - Device code: code runs on GPU

Review: CPU Program

```
void vecAdd(float *a, float *b, float *c,  
            size_t length) {  
    for (size_t i=0; i<length; i++)  
        c[i] = a[i] + b[i];  
}
```

1. Define function

```
int main() {  
    float *a, *b, *c;  
    size_t length = 1024;  
    malloc(&a, length * sizeof(float));  
    ... // malloc for b and c  
    load_input(a, b, length);  
    vecAdd(a, b, c, length);  
    save_output(c, length);  
    free(a);  
    ... // free for b and c  
    return 0;  
}
```

2. Allocate CPU memory

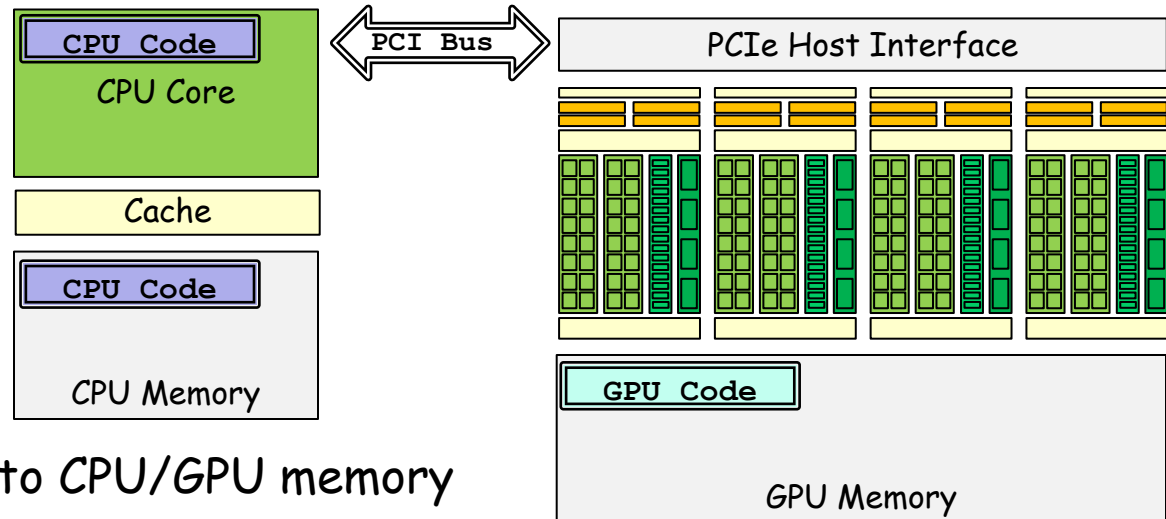
3. Load Data

4. Do computation

5. Save Result

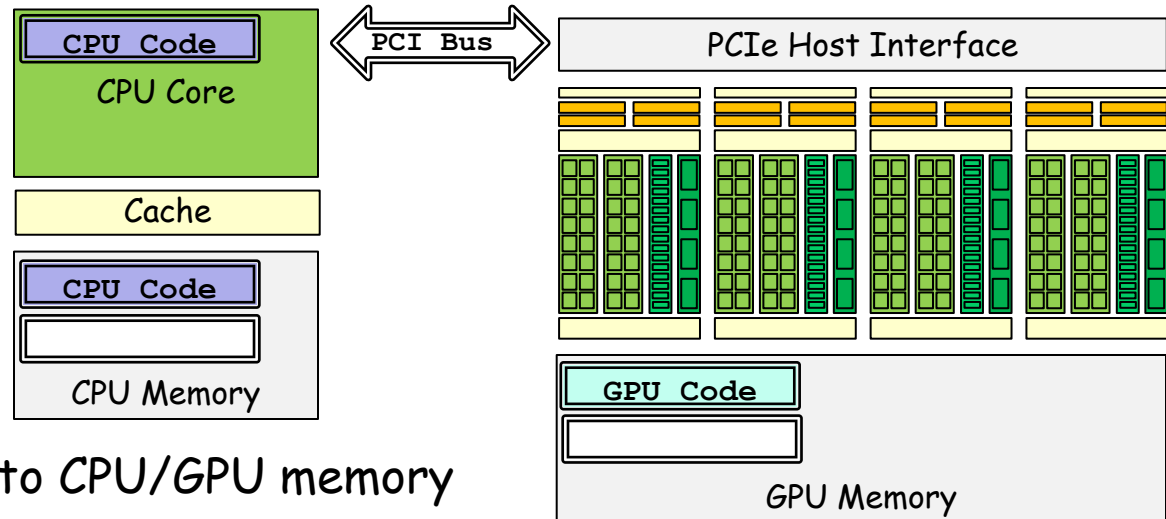
6. Release CPU memory

GPU Program Workflow



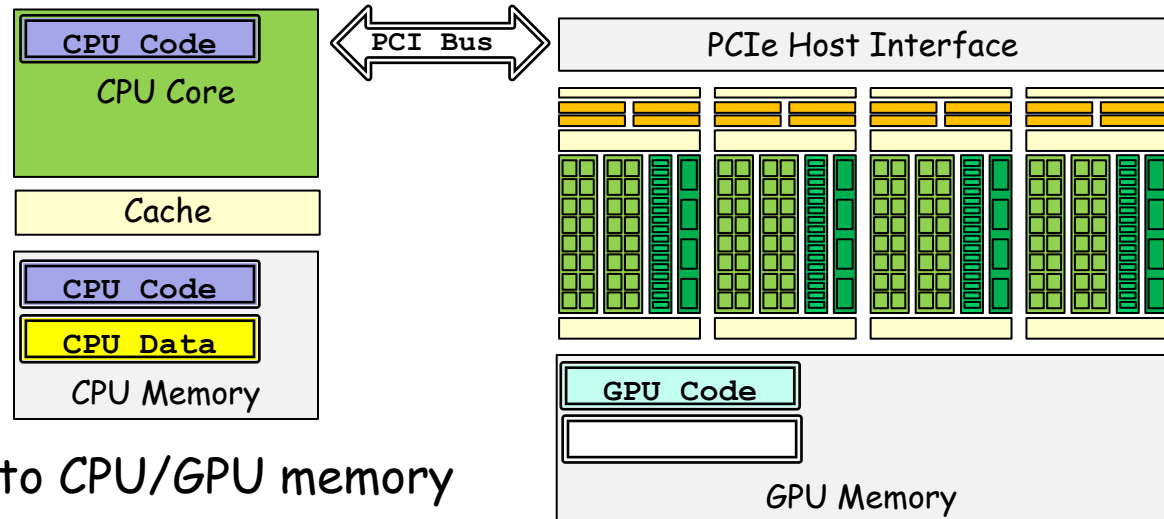
1. Load CPU/GPU code to CPU/GPU memory

GPU Program Workflow



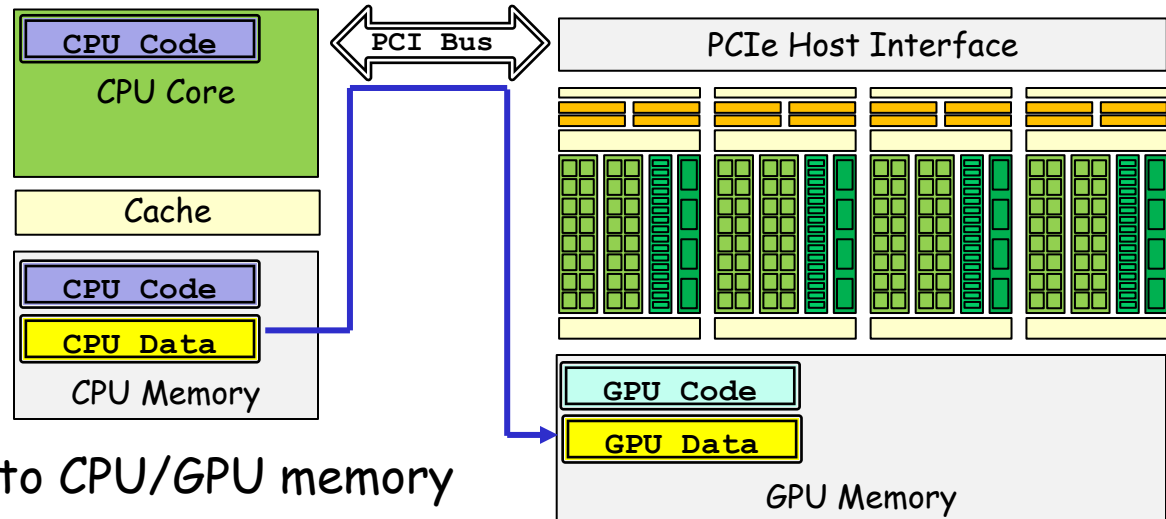
1. Load CPU/GPU code to CPU/GPU memory
2. Allocate CPU/GPU memory for data

GPU Program Workflow



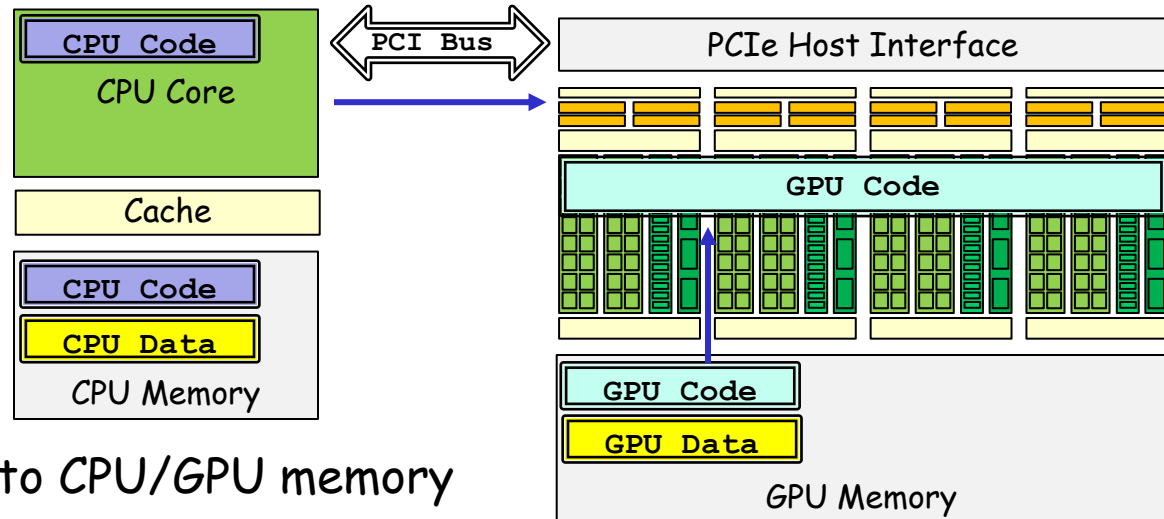
1. Load CPU/GPU code to CPU/GPU memory
2. Allocate CPU/GPU memory for data
3. Load data to CPU memory

GPU Program Workflow



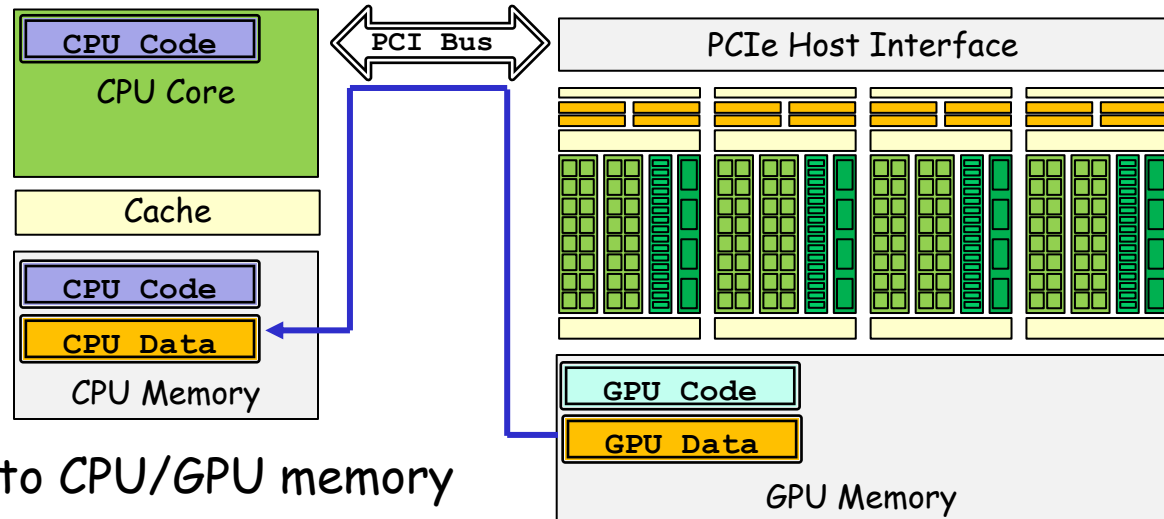
1. Load CPU/GPU code to CPU/GPU memory
2. Allocate CPU/GPU memory for data
3. Load data to CPU memory
4. Copy data from CPU memory to GPU memory

GPU Program Workflow



1. Load CPU/GPU code to CPU/GPU memory
2. Allocate CPU/GPU memory for data
3. Load data to CPU memory
4. Copy data from CPU memory to GPU memory
5. Start GPU code execution

GPU Program Workflow



1. Load CPU/GPU code to CPU/GPU memory
2. Allocate CPU/GPU memory for data
3. Load data to CPU memory
4. Copy data from CPU memory to GPU memory
5. Start GPU code execution
6. Copy data from GPU memory to CPU memory

Define function on GPU

- GPU Kernel: a function runs on GPU
 - The keyword `__global__` indicates a function that
 - Runs on the device
 - Is called from the host code
 - Is written from the point of view of **a single thread**

```
__global__ void vecAdd(float *a, float *b, float *c, size_t length)
{
    size_t idx = blockDim.x * blockIdx.x + threadIdx.x;
    if (idx < length) {
        c[idx] = a[idx] + b[idx];
    }
}
```

SPMD Programming Model

- Single-Program-Multi-Data
 - All processors execute the same program
 - In CUDA:
 - Processors: CUDA cores
 - The program: CUDA kernel (e.g., vecAdd)

Allocate GPU memory

- Host and device memory are separate entities
 - Device pointers point to GPU memory
 - May be passed to/from host code
 - May not be dereferenced in host code
 - Host pointers point to CPU memory
 - May be passed to/from device code
 - May not be dereferenced in device code
- Simple CUDA API for handling device memory
 - `cudaMalloc()`, `cudaFree()`
 - Similar to `malloc()` and `free()`

```
float *h_A; // host pointer
h_A = (float *)malloc(size);
float *d_A; // device pointer
cudaMalloc((void **)&d_A, size);
```

Load Data to GPU

- Use `cudaMemcpy()` to copy data between host and device
 - Similar to `memcpy()`, but with directions
 - `cudaMemcpyHostToDevice`
 - `cudaMemcpyDeviceToHost`

```
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);  
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
```

Do Computation on GPU

- “<<< >>>” marks a call from host code to device code
 - Also called a “kernel launch”
 - Vars in <<< >>> indicates the parallelism
 - Vars in () are kernel arguments

```
vectorAdd<<<16, 64>>>(d_A, d_B, d_C, length);
```

CUDA Threading Model

- CUDA programs consist of a hierarchy of concurrent threads
 - Grid - Block - Thread
 - A grid consists of multiple blocks
 - A block consists of multiple threads
 - Launch a kernel means launches a grid of blocks

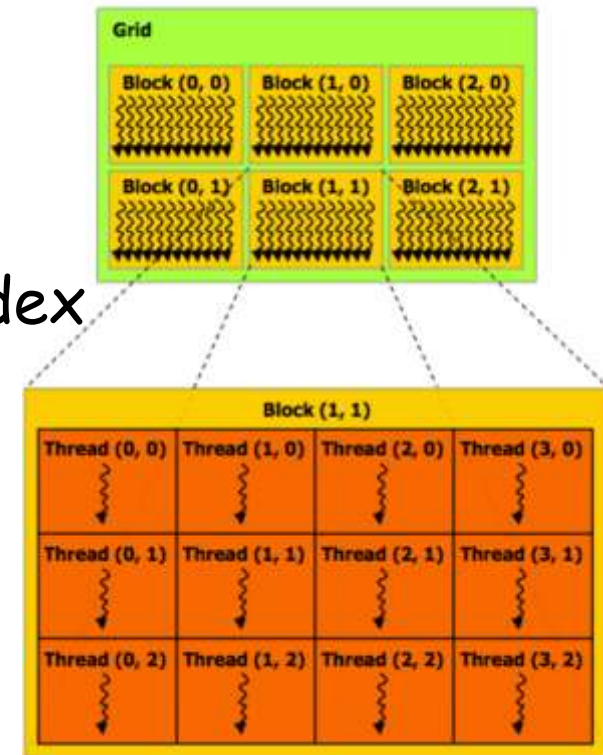
```
vectorAdd<<<16, 64>>>>(d_A, d_B, d_C, length);
```

16 blocks

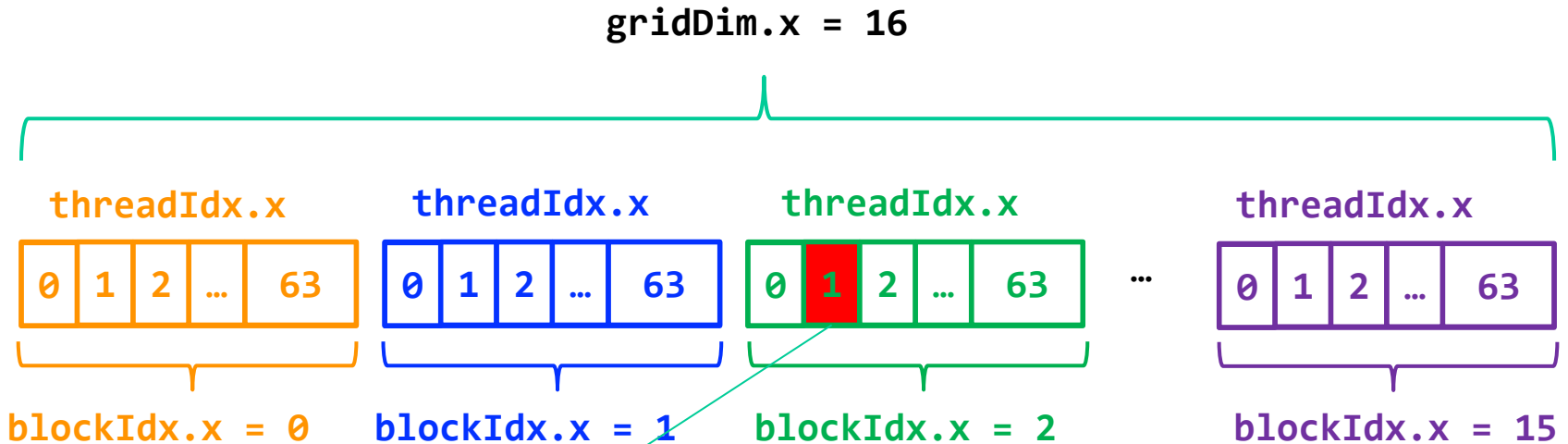
64 threads per block

CUDA Threading Model

- Grid/Block also can be up to 3-dimensional
 - 1-dimensional at default
 - `gridDim.x`, `gridDim.y`, `gridDim.z`
 - `blockDim.x`, `blockDim.y`, `blockDim.z`
- Each Block/Thread has its unique index
 - Each block can be identified by
 - `blockIdx.x`, `blockIdx.y`, `blockIdx.z`
 - Each thread can be identified by
 - `blockIdx.x`, `blockIdx.y`, `blockIdx.z`
 - `threadIdx.x`, `threadIdx.y`, `threadIdx.z`



Indexing Arrays with Blocks and Threads



`total_threads = gridDim.x * blockDim.x`

`total_threads = (16) * (64)`

`thread_id = blockDim.x * blockIdx.x + threadIdx.x`

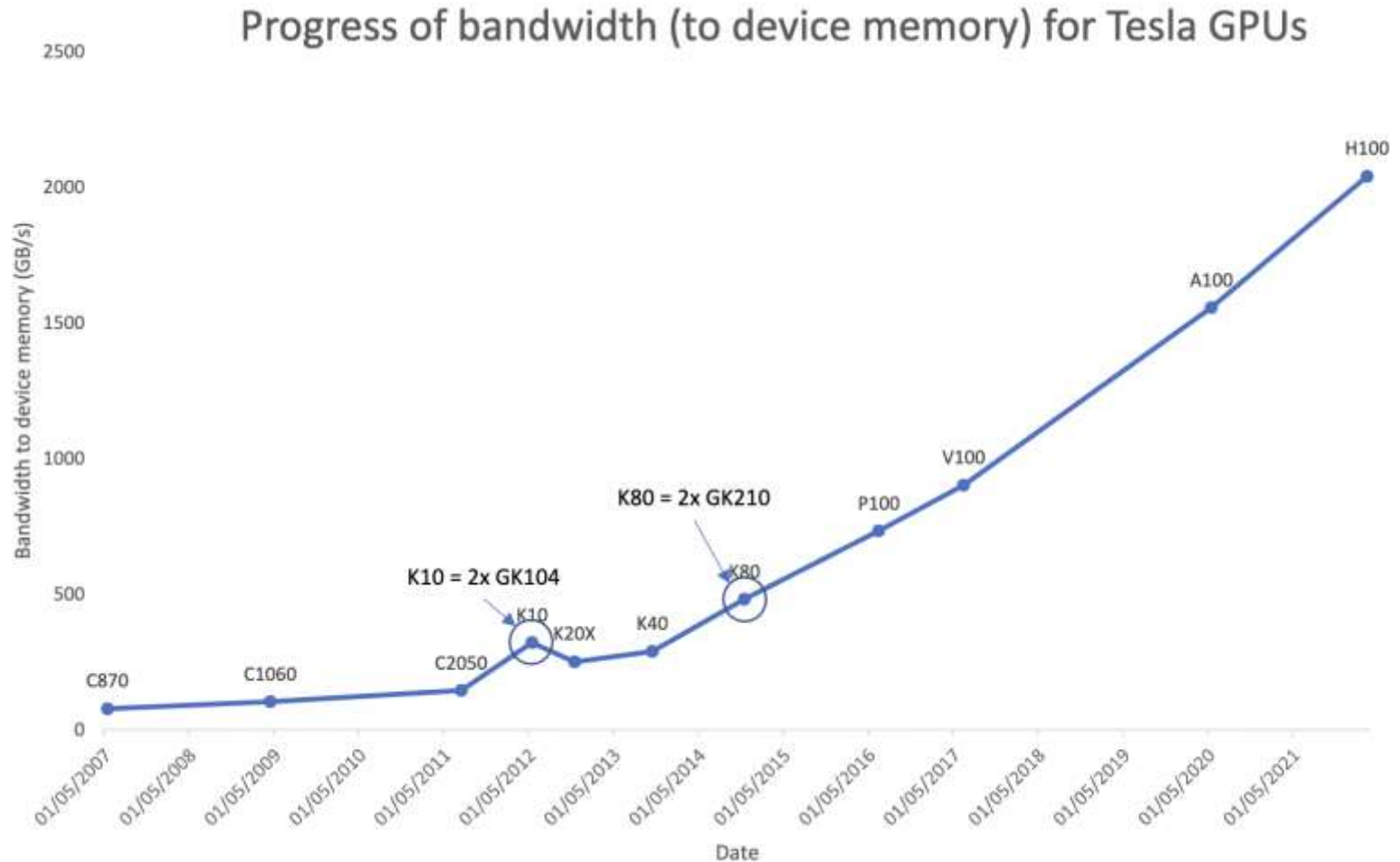
`thread_id = (64) * (2) + (1)`

Mapping CUDA Threads to CUDA Cores

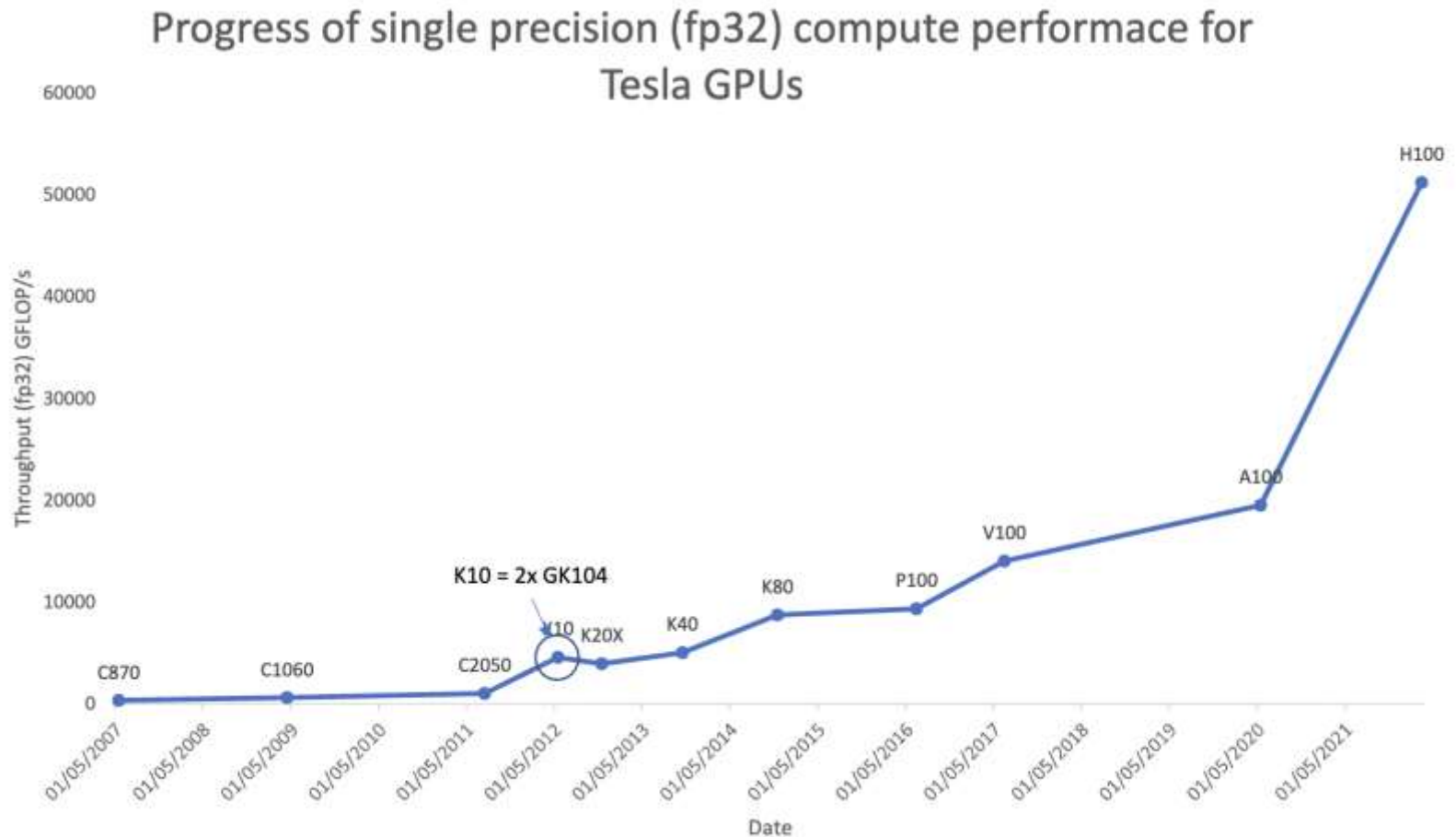
- All threads in a block runs on the same SM
 - Never split, never move to other SMs
- An SM can execute multiple blocks concurrently
- Blocks are automatically assigned to SMs



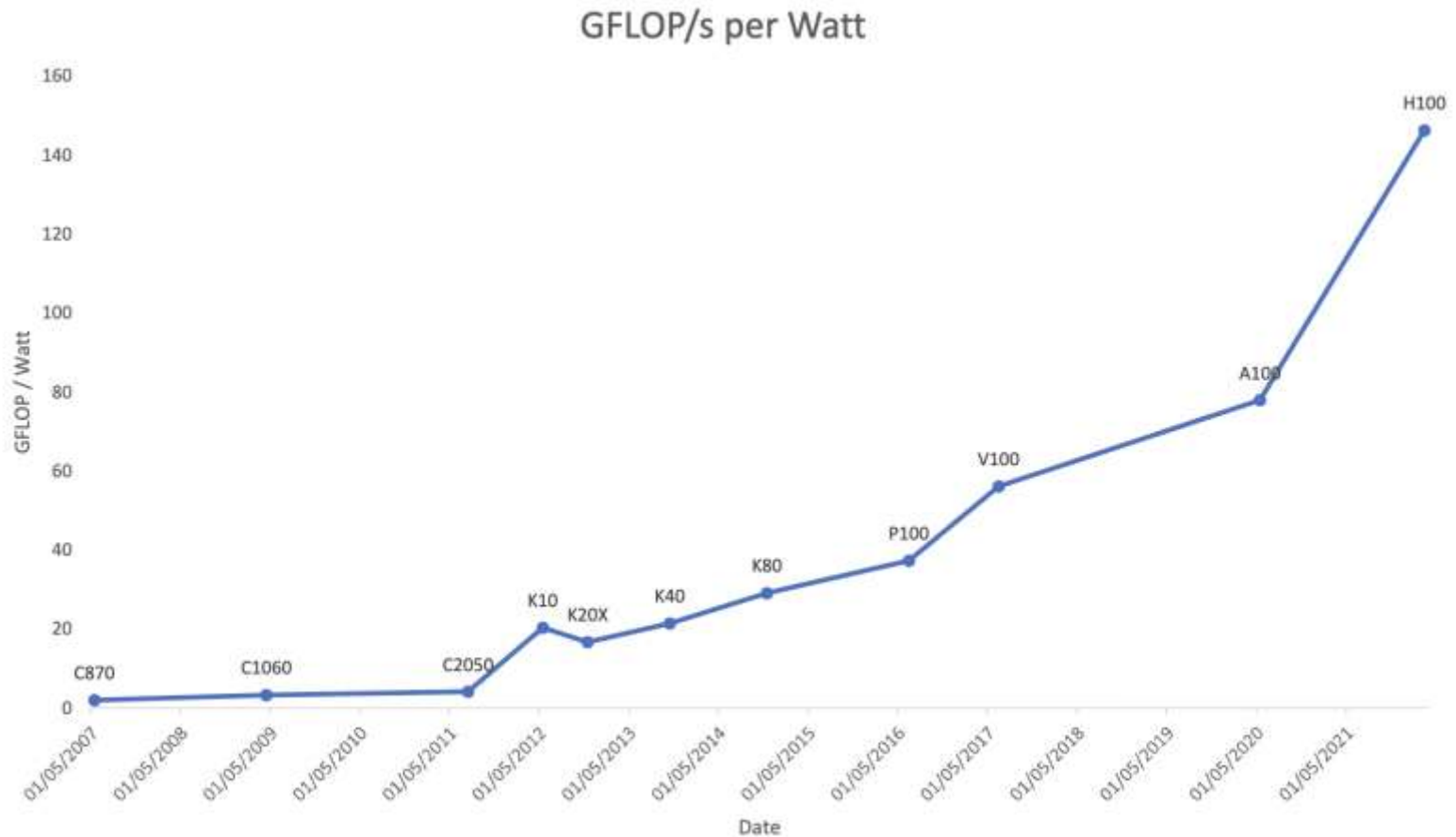
Trends: Bandwidth



Trends: Compute

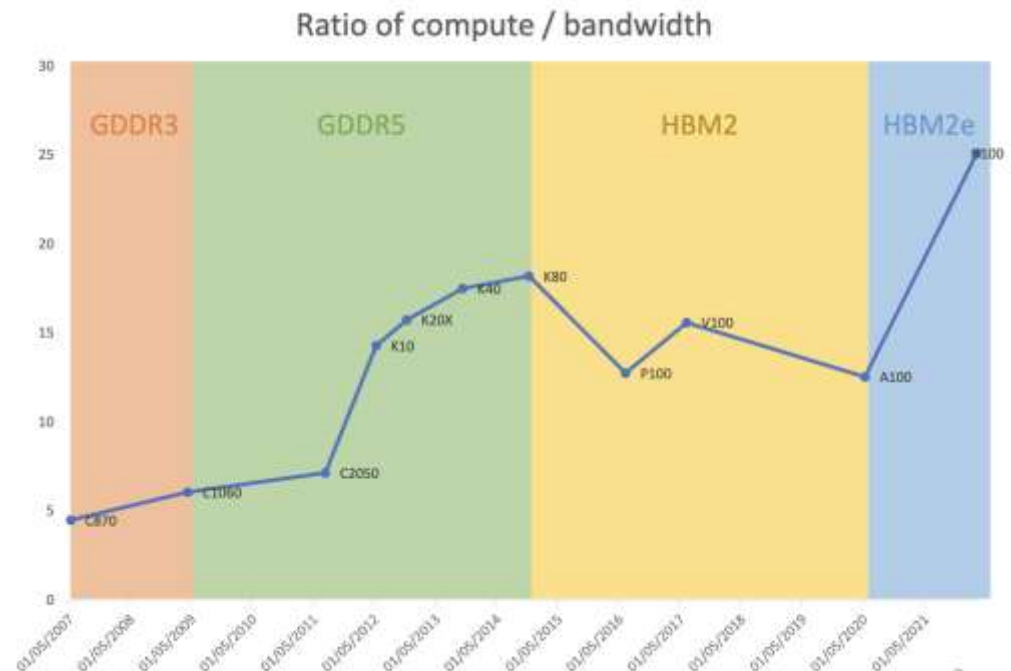


Trends: Efficiency



Ratio of (Peak) Compute / Bandwidth

- The ratio of compute / bandwidth often called arithmetic intensity or operational intensity (I)
 - how many floating point operations we can perform in the time it takes to move each byte of data from device memory
- The overall trend is for it to increase



Roofline Model

- The roofline model tells us, for given hardware, whether our application will be **bandwidth bound** or **compute bound**

